

Universidade Federal do Ceará
Campus Quixadá
QXD0068 – Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

Trabalho 1
Refatoração com Padrões de Projeto em Sistema Legado

Daniel Lucas Ulisses Magalhães - 554563

Julian Cesar Pereira Cardoso - 553782

Quixadá-CE
2026

Sumário

1. Introdução..... 2
2. Caracterização e Diagnóstico do Sistema Legado.....3
2.1 Caracterização do Sistema.....3
2.2 Problemas Identificados.....3
3. Padrões de Projeto Selecionados.....3
3.1 Strategy.....4
3.2 Observer..... 4
3.3 Repository.....5
3.3 Factory (Factory Method).....6
4. Refatorações Realizadas..... 7
4.1 Padrão 1 - Strategy.....7
4.2 Padrão 2 - Observer..... 13
4.3 Padrão 3 – Repository..... 16
4.4 Padrão 4 – Factory..... 19
5. Análise dos Resultados..... 21
5.1 Strategy e Observer.....21
5.2 Factory e Repository.....23
6. Conclusão..... 24
7. Referências..... 25

1. Introdução

A manutenção e a evolução de sistemas legados de software representam desafios recorrentes na engenharia de software. Com o crescimento e a alteração contínua de requisitos sem o devido cuidado estrutural, o código-fonte tende a degradar-se, manifestando elevado acoplamento, baixa coesão e proliferação de code smells. A refatoração orientada por padrões de projeto GoF (Gang of Four) surge como uma prática essencial para reestruturar o código sem alterar o seu comportamento externo, elevando os níveis de reusabilidade, manutenibilidade e extensibilidade.

O sistema selecionado para este trabalho é uma aplicação web de e-commerce e gestão administrativa de eletrónicos denominada Electronics eCommerce Shop with Admin Dashboard, construída sobre o ecossistema Next.js, Node.js e Prisma ORM. A escolha deste sistema justifica-se pela sua arquitetura real de médio porte, que apresenta complexidade suficiente para evidenciar problemas concretos de projeto no tratamento de dados, regras de negócio e disparo de notificações.

O objetivo principal deste trabalho consiste em diagnosticar problemas estruturais e de design no sistema original, propor e implementar refatorações utilizando padrões GoF e o Repository e analisar empiricamente os ganhos de qualidade de software obtidos.

Link do repositório original

Link do fork do repositório: onde estão as implementações

2. Caracterização e Diagnóstico do Sistema Legado

2.1 Caracterização do Sistema

O Electronics eCommerce Shop é uma plataforma e-commerce voltada para a comercialização de produtos eletrônicos. O sistema está dividido em duas frentes principais:

- **Loja do Cliente (Storefront)**: navegação por produtos e categorias, pesquisa/filtragem, gestão do carrinho de compras, lista de desejos (wishlist) e finalização de pedidos (checkout).
- **Painel de Administração (Admin Dashboard):** gestão CRUD de utilizadores, produtos, categorias, pedidos e carga massiva de dados via CSV (bulk upload).

Tecnologias e Estrutura:

- **Frontend/Backend:** Next.js (React / App Router), TypeScript, Tailwind CSS.
- **Persistência de Dados e ORM:** PostgreSQL / SQLite com Prisma ORM.
- **Gestão de Estado:** Zustand e React Hooks.

2.2 Problemas Identificados

A análise inicial do código-fonte revelou fragilidades estruturais que dificultam a manutenção e a reutilização:

- **Condicionais Excessivas e Lógica de Pedidos Rígida (Switch Statements / Smells):** a gestão das regras de estado e processamento de pedidos continha ramificações condicionais extensas em blocos imperativos no controlador/módulo de orders, dificultando a adição de novos fluxos de liquidação.
- **Acoplamento Forte e Disparo Direto de Notificações:** sempre que o estado de um pedido mudava ou um utilizador realizava uma ação, as notificações e alertas eram invocados diretamente dentro dos manipuladores do componente/serviço, violando o princípio da responsabilidade única (SRP).
- **Invocação Espalhada e Repetitiva de Sanitização e Validação:** funções de sanitize e limpeza de inputs encontravam-se dispersas pela aplicação (lib/sanitize.ts, lib/form-sanitize.ts), gerando duplicação e inconsistências de segurança.
- **Criação Descentralizada de Serviços de API:** a instanciação de instâncias e clientes de API/notificação era realizada de forma não centralizada, aumentando o consumo desnecessário de recursos

Apresentar **evidências**, utilizando trechos de código, diagramas ou outras informações relevantes.

3. Padrões de Projeto Selecionados

Apresentar **no mínimo três padrões de projeto GoF** utilizados na refatoração.

Para solucionar os problemas identificados, foram selecionados três padrões de projeto GoF(Strategy, Observer e Factory Method), complementados pelo padrão Repository para a camada de acesso a dados

3.1 Strategy

- **Categoria:** comportamental
- **Problema identificado no sistema:** Na página de checkout (app/checkout/page.tsx), a função makePurchase concentrava, em um único manipulador de cerca de 200 linhas, a validação do formulário, o envio da requisição POST /api/orders, a interpretação manual das respostas de erro (409 de pedido duplicado, erros de validação e falhas não estruturadas) e o loop de inclusão dos itens com addOrderProduct. Esse acoplamento entre a camada de interface e o protocolo de comunicação com a API impedia substituir ou variar a forma de criação do pedido sem reescrever o componente e tornava essa lógica impossível de testar isoladamente. A busca do userId e a montagem do orderData permaneceram na página, fora do escopo desta refatoração.
- **Justificativa para utilização:** o padrão Strategy permite isolar o algoritmo e os passos de execução da criação do pedido atrás de uma interface coesa. Mover a orquestração da requisição para uma classe de estratégia (OrderStrategy.ts) permite que o componente React apenas invoque a estratégia selecionada. Isso possibilita alterar ou criar novas formas de processamento (como um checkout via GraphQL, uma estratégia Offline First salvando localmente ou um Mock Checkout para testes automatizados) sem poluir a página com blocos condicionais (if/else) complexos.
- **Classes ou comportamentos envolvidos:**
    - A estratégia concreta **RestApiOrderStrategy**, que implementa **OrderCreationStrategy**, encapsula a chamada POST /api/orders, o tratamento dos erros HTTP (409 de pedido duplicado, 400 de validação e falhas genéricas) e o loop de inclusão dos produtos do pedido, por meio da função addOrderProductFn. A página de checkout (app/checkout/page.tsx) atua como cliente do padrão: valida o formulário, monta o orderData e delega a criação do pedido ao **OrderContext**, que executa a estratégia configurada.
- **Resultado esperado com a aplicação:** desacoplamento entre a página de checkout e a camada de integração HTTP de criação do pedido, facilitando a inclusão de novas formas de processamento de pedidos e tornando o código do frontend imune a alterações nas regras de comunicação.

3.2 Observer

- **Categoria:** comportamental
- **Problema identificado no sistema:** após a conclusão de um pedido na página de checkout (app/checkout/page.tsx), o sistema precisava notificar outras partes da aplicação (como atualizar o contador de notificações global da interface). Na abordagem original, o evento era disparado imperativamente dentro do manipulador do formulário através de um bloco try/catch com window.dispatchEvent(new CustomEvent('orderCompleted')). Isso gerava um acoplamento direto entre o fluxo de finalização de compra e a implementação concreta do evento de janela do browser, além de dificultar o registro de múltiplos reatores (como envio de e-mails, atualização de analytics ou emissão de logs) sem alterar o código principal.
- **Justificativa para utilização:** o padrão Observer estabelece uma relação de dependência um-para-muitos entre objetos, garantindo que quando o estado do objeto central (Sujeito) muda, todos os seus dependentes (Observadores) são notificados e atualizados automaticamente. A utilização deste padrão desacopla a conclusão do pedido

(makePurchase) das ações colaterais resultantes. Desta forma, é possível cadastrar dinamicamente novos observadores (por exemplo, observadores para auditoria, e-mail ou telemetria) chamando apenas orderSubject.subscribe(...), sem precisar modificar a função makePurchase ou o fluxo da página de checkout.

- **Classes ou comportamentos envolvidos:**
  - **OrderObserver** (Interface - lib/observer/orderObserver.ts): contrato que define o método onOrderCompleted(orderId: string) que todo observador deve implementar.
  - **WindowEventObserver e LoggerObserver** (Observadores Concreto - lib/observer/orderObserver.ts): Classe que encapsula o disparo do evento customizado do navegador (orderCompleted) com o tratamento de exceção adequado e registra o pedido concluído no console (auditoria) e demonstra o ganho de extensibilidade: um novo observador foi adicionado sem alterar makePurchase
  - **OrderSubject** (Sujeito Observável - lib/observer/orderObserver.ts): Classe gerenciadora responsável por manter a lista de observadores (observers), prover métodos para inscrição/cancelamento (subscribe/unsubscribe) e emitir notificações globais (notifyAll).
  - **orderSubject** (Instância Singleton - lib/observer/orderObserver.ts): Instância exportada e pré-configurada onde os observadores concretos se registram.
  - **OrderSubject.notifyAll:** sola cada observador em um try/catch, para que a falha de um não impeça a notificação dos demais.
  - **hooks/useNotifications.ts** (listener de orderCompleted): atualiza o contador de notificações.
  - **CheckoutPage** (Contexto/Emissor - app/checkout/page.tsx): Componente de interface que invoca orderSubject.notifyAll(orderId) após a criação bem-sucedida da ordem.
- **Resultado esperado com a aplicação:** isolamento completo do fluxo de encerramento da compra em relação às ações colaterais da aplicação. Facilidade para estender o sistema registrando múltiplos observadores sem alterar nenhuma linha do componente React.

3.3 Repository

- **Categoria:** Acesso a dados
- **Problema identificado no sistema:** nos controllers do backend (server/controllers/product.js e server/controllers/costumer_orders.js), todo o acesso ao banco era feito com chamadas diretas ao Prisma (prisma.product.findMany, prima.costumer_order.create com os 13 campos repetidos a cada chamada, calculo de paginação e regra anti-duplicidade escritos no meio do controller). Além disso, cada módulo criava o seu próprio cliente com new PrismaClient() (costumer_order.js, notificationController.js e notificationHelpers.js), mantendo vários pools de conexão abertos simultaneamente em vez de usar a conexão compartilhada de server/utills/db.js. Essa dispersão impedia testar os controller sem banco de dados, duplica a lógica de consulta e faria qualquer troca de ORM exigir reescrita de todos os controllers
- **Justificativa para utilização:** o padrão Repository medeia o acesso a dados atrás de uma interface coesa: o controller declara o quê deseja ("busque o pedido pelo id") e o repository resolve como (tabela, `where`, `include`, paginação). Com isso, os controllers passam a depender da abstração e não do Prisma; a troca de ORM fica restrita aos repositórios; e a injeção do cliente Prisma pelo construtor permite testar toda a camada com um objeto fake, sem banco.

- **Classe ou comportamentos envolvidos:**
    - **ProductRepository** (`server/repositories/ProductRepository.js`): concentra todas as consultas e escritas de produtos: listagem com filtros, paginação e ordenação, busca por id ou slug, busca textual e a verificação de itens vinculados em `customer_order_product` antes de permitir o delete —, escondendo dos controllers os detalhes de `where`, `include` e `skip`/`take` do Prisma. Recebe o cliente Prisma pelo construtor, o que permite testá-lo com um objeto fake, sem banco.
    - **OrderRepository (`server/repositories/OrderRepository.js`):** encapsula a paginação de pedidos, a regra anti-duplicidade (mesmo e-mail e mesmo total dentro de 1 minuto) e a criação e atualização com filtragem de campos vazios, de modo que o controller apenas entrega os dados já validados e recebe o pedido pronto.
    - **NotificationRepository e UserRepository (criados nesta refatoração):** o primeiro isola a persistência de notificações e é usado tanto pela Factory quanto pelo controller de notificações; o segundo elimina os `prisma.user.findUnique` espalhados pelos controllers, centralizando a localização do usuário por id ou e-mail.
    - **repositories/index.js (ponto central de exportação) e os controllers clientes (`products.js`, `customer_orders.js` e `notificationController.js`):** os controllers atuam como clientes do padrão — pedem dados aos repositórios sem conhecer o Prisma — de modo que uma eventual troca de ORM ficaria restrita aos quatro arquivos de repositório.
- **Resultado esperado com a aplicação:** controllers sem nenhuma chamada direta ao Prisma; acesso a dados centralizado, reutilizável e testável com mocks; troca de ORM restrita a quatro arquivos.

3.3 Factory (Factory Method).

- **Categoria:** criacional (GoF).
- **Problema identificado no sistema:** a criação de notificações estava dispersa em cinco funções independentes em `server/utills/notificationHelpers.js` (~247 linhas), cada uma com a sua própria cópia do mapa de mensagens por status, da geração de id (com `await import('nanoid')` e fallback para `Math.random`) e da chamada `prisma.notification.create` — inclusive com um `new PrismaClient()` próprio. Adicionar um novo tipo de notificação exigia copiar mais um bloco, e qualquer mudança no formato (ex.: validação de prioridade) precisava ser repetida em cinco lugares.
- **Justificativa para utilização**: o Factory Method centraliza a criação de objetos num único ponto e encapsula as variações por tipo atrás de um despachante (`create(type, payload)`). Separar a **montagem** do objeto (métodos `build*`, puros, sem I/O) da **persistência** (métodos `create*`, que delegam ao `NotificationRepository`) permite testar a regra de montagem sem banco e trocar a persistência sem tocar na regra.
- **Classes ou comportamentos envolvidos:**
    - **NotificationFactory (`server/factories/NotificationFactory.js`):** classe estática que centraliza a criação de todos os tipos de notificação a partir de mapas únicos de mensagens por status, separando a montagem do objeto (métodos `build*`, puros e testáveis sem banco, com IDs gerados por `crypto.randomUUID()` e validação central de obrigatoriedade e enums) da persistência (métodos `create*`, que delegam ao `NotificationRepository` sem importar o Prisma). Expõe ainda o despachante genérico

`create(type, payload)` como ponto único de entrada e o método `setRepository()` para injeção de repositório fake nos testes, de modo que um tipo novo de notificação exige apenas um novo `build*` e um novo `case` no despachante.

    - **factories/index.js (exportação central da fábrica) e os consumidores:** o `notificationHelpers.js` atua como fachada de compatibilidade — cada função apenas delega à fábrica, preservando a API antiga — e os controllers (como `customer_orders.js`, via helpers) são os clientes do padrão, solicitando notificações sem conhecer sua montagem nem sua persistência.
- **Resultado esperado com a aplicação:** ponto único de criação de notificações; um tipo novo exige apenas um `build*` + um `case` no despachante; consistência de IDs, mensagens e prioridades; eliminação das ~200 linhas duplicadas dos helpers.

4. Refatorações Realizadas

4.1 Padrão 1 - Strategy

- **Situação original:** a função makePurchase no arquivo app/checkout/page.tsx realizava a montagem completa do objeto orderData limpando strings com .trim(), buscava o userId em endpoint auxiliar, chamava a rota /api/orders com apiClient.post, tratava manualmente códigos de erro HTTP (como 409 e 400), iterava no array products chamando addOrderProduct(orderId, ...) para cada item e manipulava o estado da aplicação.
- **Problema Identificado:** na página de checkout (app/checkout/page.tsx), a função makePurchase concentrava, em um único manipulador de cerca de 200 linhas (linhas 90 a 298 do arquivo original), responsabilidades de natureza distinta: validação do formulário, envio da requisição POST /api/orders por meio do apiClient, interpretação manual das respostas de erro (status 409 de pedido duplicado, erros de validação em formato de lista ou texto, falhas não estruturadas), leitura do JSON de resposta e o loop de inclusão dos itens com addOrderProduct. Esse acoplamento entre a camada de interface e o protocolo de comunicação com a API impedia substituir ou variar a forma de criação do pedido (por exemplo, por outro protocolo ou por um mock em testes automatizados) sem reescrever o componente de checkout, além de tornar a lógica de criação impossível de testar isoladamente. A busca do userId e a montagem do orderData permaneceram na página, fora do escopo desta refatoração.
- **Solução Implementada:** a criação do pedido (requisição POST /api/orders, tratamento de erros HTTP e inclusão dos produtos com addOrderProduct) foi movida para o método createOrder de RestApiOrderStrategy. Em app/checkout/page.tsx, a função makePurchase mantém as validações de interface, a busca do userId e a montagem do orderData, e delega a criação do pedido a OrderContext.executeOrderCreation. Os payloads receberam tipos explícitos (OrderData, OrderProductItem, OrderPayload) no lugar de any.
- **Trecho do código (ANTES):**



```
const makePurchase = async () => {
  // Client-side validation first
  const validationErrors = validateForm();
  if (validationErrors.length > 0) {
    validationErrors.forEach(error => {
      toast.error(error);
    });
    return;
  }

  // Basic client-side checks for required fields (UX only)
  const requiredFields = [
    'name', 'lastname', 'phone', 'email', 'company',
    'adress', 'apartment', 'city', 'country', 'postalCode'
  ];
```



```
setIsSubmitting(true);

try {
  console.log("🚀 Starting order creation...");

  // Get user ID if logged in
  let userId = null;
  if (session?.user?.email) {
    try {
      console.log("🔍 Getting user ID for logged-in user:", session.user.email);
      const userResponse = await apiClient.get(`/api/users/email/${session.user.email}`);
      if (userResponse.ok) {
        const userData = await userResponse.json();
        userId = userData.id;
        console.log("✅ Found user ID:", userId);
      } else {
        console.log("❌ Could not find user with email:", session.user.email);
      }
    } catch (userError) {
      console.log("⚠️  Error getting user ID:", userError);
    }
  }

  // Prepare the order data
  const orderData = {
    name: checkoutForm.name.trim(),
    lastname: checkoutForm.lastname.trim(),
    phone: checkoutForm.phone.trim(),
    email: checkoutForm.email.trim().toLowerCase(),
    company: checkoutForm.company.trim(),
    adress: checkoutForm.adress.trim(),
    apartment: checkoutForm.apartment.trim(),
    postalCode: checkoutForm.postalCode.trim(),
    status: "pending",
    total: total,
    city: checkoutForm.city.trim(),
```



```
  country: checkoutForm.country.trim(),
  orderNotice: checkoutForm.orderNotice.trim(),
  userId: userId // Add user ID for notifications
};

console.log("📦 Order data being sent:", orderData);

// Send order data to server for validation and processing
const response = await apiClient.post("/api/orders", orderData);

console.log("🎉 API Response received:");
console.log("  Status:", response.status);
console.log("  Status Text:", response.statusText);
console.log("  Response OK:", response.ok);

// Check if response is ok before parsing
if (!response.ok) {
  console.error("❌ Response not OK:", response.status, response.statusText);
  const errorText = await response.text();
  console.error("Error response body:", errorText);

  // Try to parse as JSON to get detailed error info
  try {
    const errorData = JSON.parse(errorText);
    console.error("Parsed error data:", errorData);

    // Handle different error types
    if (response.status === 409) {
      // Duplicate order error
      toast.error(errorData.details || errorData.error || "Duplicate order detected");
      return; // Don't throw, just return to stop execution
    } else if (errorData.details && Array.isArray(errorData.details)) {
      // Validation errors
      errorData.details.forEach((detail: any) => {
        toast.error(`${detail.field}: ${detail.message}`);
      });
```



```
    } else if (typeof errorData.details === 'string') {
      // Single error message in details
      toast.error(errorData.details);
    } else {
      // Fallback error message
      toast.error(errorData.error || "Order creation failed");
    }
  } catch (parseError) {
    console.error("Could not parse error as JSON:", parseError);
    toast.error("Order creation failed. Please try again.");
  }

  return; // Stop execution instead of throwing
}

const data = await response.json();
console.log("✅ Parsed response data:", data);

const orderId: string = data.id;
console.log("🆔 Extracted order ID:", orderId);

if (!orderId) {
  console.error("❌ Order ID is missing or falsy!");
  console.error("Full response data:", JSON.stringify(data, null, 2));
  throw new Error("Order ID not received from server");
}

console.log("✅ Order ID validation passed, proceeding with product addition...");

// Add products to order
for (let i = 0; i < products.length; i++) {
  console.log(`📦 Adding product ${i + 1}/${products.length}:`, {
    orderId,
    productId: products[i].id,
    quantity: products[i].amount
```



```
    quantity: products[i].amount
  });

  await addOrderProduct(orderId, products[i].id, products[i].amount);
  console.log(`✅ Product ${i + 1} added successfully`);
}

console.log("🎉 All products added successfully!");

// Clear form and cart
setCheckoutForm({
  name: "",
  lastname: "",
  phone: "",
  email: "",
  company: "",
  adress: "",
  apartment: "",
  city: "",
  country: "",
  postalCode: "",
  orderNotice: "",
});
clearCart();

// Refresh notification count if user is logged in
try {
  // This will trigger a refresh of notifications in the background
  window.dispatchEvent(new CustomEvent('orderCompleted'));
} catch (error) {
  console.log('Note: Could not trigger notification refresh');
}

toast.success("Order created successfully! You will be contacted for payment.");
setTimeout(() => {
  router.push("/");
```



```
      router.push("/");
    }, 1000);
  } catch (error: any) {
    console.error("💥 Error in makePurchase:", error);

    // Handle server validation errors
    if (error.response?.status === 400) {
      console.log("🔄 Handling 400 error...");
      try {
        const errorData = await error.response.json();
        console.log("Error data:", errorData);
        if (errorData.details && Array.isArray(errorData.details)) {
          // Show specific validation errors
          errorData.details.forEach((detail: any) => {
            toast.error(`${detail.field}: ${detail.message}`);
          });
        } else {
          toast.error(errorData.error || "Validation failed");
        }
      } catch (parseError) {
        console.error("Failed to parse error response:", parseError);
        toast.error("Validation failed");
      }
    } else if (error.response?.status === 409) {
      toast.error("Duplicate order detected. Please wait before creating another order.");
    } else {
      console.log("🔍 Handling generic error...");
      toast.error("Failed to create order. Please try again.");
    }
  } finally {
    setIsSubmitting(false);
  }
};
```

Trecho do código (DEPOIS)

```
const makePurchase = async () => {
  const validationErrors = validateForm();
  if (validationErrors.length > 0) {
    validationErrors.forEach((error) => toast.error(error));
    return;
  }

  const requiredFields = [
    "name", "lastname", "phone", "email", "company",
```

```
      "adress", "apartment", "city", "country", "postalCode",
    ];

    const missingFields = requiredFields.filter(
      (field) => !checkoutForm[field as keyof typeof checkoutForm]?.trim()
    );

    if (missingFields.length > 0) {
      toast.error("Please fill in all required fields");
      return;
    }

    if (products.length === 0) {
      toast.error("Your cart is empty");
      return;
    }

    if (total <= 0) {
      toast.error("Invalid order total");
      return;
    }

    setIsSubmitting(true);

    try {
      let userId = null;
      if (session?.user?.email) {
        try {
                                                const     userResponse     =     await
apiClient.get(`/api/users/email/${session.user.email}`);
          if (userResponse.ok) {
            const userData = await userResponse.json();
            userId = userData.id;
          }
        } catch (userError) {
          console.log("⚠️ Error getting user ID:", userError);
        }
      }

      const orderData = {
        name: checkoutForm.name.trim(),
        lastname: checkoutForm.lastname.trim(),
        phone: checkoutForm.phone.trim(),
        email: checkoutForm.email.trim().toLowerCase(),
        company: checkoutForm.company.trim(),
        adress: checkoutForm.adress.trim(),
        apartment: checkoutForm.apartment.trim(),
        postalCode: checkoutForm.postalCode.trim(),
        status: "pending",
        total: total,
        city: checkoutForm.city.trim(),
        country: checkoutForm.country.trim(),
        orderNotice: checkoutForm.orderNotice.trim(),
        userId: userId,
      };

      // AQUI IMPLEMENTEI O STRATEGY
      const orderContext = new OrderContext(new RestApiOrderStrategy());
      const orderId = await orderContext.executeOrderCreation({
        orderData,
        products,
        addOrderProductFn: addOrderProduct,
      });

      // Clear form and cart
      setCheckoutForm({
        name: "",
        lastname: "",
        phone: "",
        email: "",
```

```
        company: "",
        adress: "",
        apartment: "",
        city: "",
        country: "",
        postalCode: "",
        orderNotice: "",
      });
      clearCart();
```

Strategy

```
import apiClient from "@/lib/api";
import toast from "react-hot-toast";

//tipos de dados
export interface OrderData {
  name: string;
  lastname: string;
  phone: string;
  email: string;
  company: string;
  adress: string;
  apartment: string;
  postalCode: string;
  status: string;
  total: number;
  city: string;
  country: string;
  orderNotice: string;
  userId: string | null;
}

export interface OrderProductItem {
  id: string;
  amount: number;
}

//interface de dados
export interface OrderPayload {
  orderData: OrderData;
  products: OrderProductItem[];
   addOrderProductFn: (orderId: string, productId: string, quantity: number) =>
Promise<void>;
}

//interface da strategy
export interface OrderCreationStrategy {
  createOrder(payload: OrderPayload): Promise<string>;
}
```

```
// strategy
export class RestApiOrderStrategy implements OrderCreationStrategy {
  async createOrder({ orderData, products, addOrderProductFn }: OrderPayload):
Promise<string> {
    console.log("starting order creation via RestApiOrderStrategy...");

    const response = await apiClient.post("/api/orders", orderData);

    if (!response.ok) {
      const errorText = await response.text();
      try {
        const errorData = JSON.parse(errorText);
        if (response.status === 409) {
           toast.error(errorData.details || errorData.error || "Duplicate order
detected");
          throw new Error("DUPLICATE_ORDER");
        } else if (errorData.details && Array.isArray(errorData.details)) {
           errorData.details.forEach((detail: { field: string; message: string
}) => {
            toast.error(`${detail.field}: ${detail.message}`);
          });
        } else if (typeof errorData.details === "string") {
          toast.error(errorData.details);
        } else {
          toast.error(errorData.error || "Order creation failed");
        }
      } catch (parseError: any) {
        if (parseError.message === "DUPLICATE_ORDER") throw parseError;
        toast.error("Order creation failed. Please try again.");
      }
      throw new Error("ORDER_CREATION_FAILED");
    }

    const data = await response.json();
    const orderId: string = data.id;

    if (!orderId) {
      throw new Error("Order ID not received from server");
    }

    for (let i = 0; i < products.length; i++) {
      await addOrderProductFn(orderId, products[i].id, products[i].amount);
    }

    return orderId;
  }
}

//context
export class OrderContext {
```

```
  private strategy: OrderCreationStrategy;

  constructor(strategy: OrderCreationStrategy) {
    this.strategy = strategy;
  }

  public setStrategy(strategy: OrderCreationStrategy) {
    this.strategy = strategy;
  }

  public async executeOrderCreation(payload: OrderPayload): Promise<string> {
    return await this.strategy.createOrder(payload);
  }
}
```

- situação original;
- problema identificado;
- solução implementada;
- trecho de código **antes e depois**;
- diagrama, quando aplicável.

4.2 Padrão 2 - Observer

- **Situação original:** ao finalizar a criação do pedido dentro do método makePurchase no componente de checkout (app/checkout/page.tsx), a atualização das notificações na interface era realizada por meio de uma chamada acoplada a eventos de DOM globais dentro de um bloco isolado try/catch

```
// Refresh notification count if user is logged in
try {
  // This will trigger a refresh of notifications in the background
  window.dispatchEvent(new CustomEvent('orderCompleted'));
} catch (error) {
  console.log('Note: Could not trigger notification refresh');
}
```

- **Problema Identificado:** a lógica da página de checkout estava ligada diretamente ao mecanismo de disparo de eventos de janela do navegador. Se surgisse a necessidade de adicionar novas reações à conclusão do pedido (como registrar auditoria, enviar logs ou atualizar um estado global de usuário), o componente visual precisaria ser editado para incluir cada uma dessas chamadas imperativamente.
- **Solução Implementada:** Foi criada a estrutura completa do padrão Observer em lib/observer/orderObserver.ts:
    1. A interface OrderObserver foi definida exigindo a assinatura do método onOrderCompleted(orderId: string).
    2. O observador concreto WindowEventObserver foi implementado para isolar o disparo do evento CustomEvent("orderCompleted", ...).
    3. A classe OrderSubject foi construída para gerenciar os observadores inscritos e notificá-los através de notifyAll(orderId).

4. Na página app/checkout/page.tsx, substituiu-se o bloco try/catch acoplado pela simples chamada do método emissor do sujeito: orderSubject.notifyAll(orderId).
5. Foi adicionado o LoggerObserver e o notifyAll passou a isolar falhas de cada observador.

- **Trecho do código (ANTES):**

Mesmo código do padrão Strategy, mas como foco nas seguintes linhas:

```
clearCart();

// Refresh notification count if user is logged in
try {
  // This will trigger a refresh of notifications in the background
  window.dispatchEvent(new CustomEvent('orderCompleted'));
} catch (error) {
  console.log('Note: Could not trigger notification refresh');
}
```

- **Trecho do código (DEPOIS):**

```
//interface do observador
export interface OrderObserver {
  onOrderCompleted(orderId: string): void;
}

//observador concreto
export class WindowEventObserver implements OrderObserver {
  onOrderCompleted(orderId: string): void {
    try {
          window.dispatchEvent(new CustomEvent("orderCompleted", { detail: { orderId } }));
    } catch (error) {
      console.log("Note: Could not trigger notification refresh");
    }
  }
}

//observador concreto 2: registro de auditoria/log
export class LoggerObserver implements OrderObserver {
  onOrderCompleted(orderId: string): void {
    console.log(`[audit] Order completed: ${orderId}`);
  }
}

//sujeito
export class OrderSubject {
```

```
  private observers: OrderObserver[] = [];

  public subscribe(observer: OrderObserver): void {
    this.observers.push(observer);
  }

  public unsubscribe(observer: OrderObserver): void {
    this.observers = this.observers.filter((obs) => obs !== observer);
  }

  public notifyAll(orderId: string): void {
    this.observers.forEach((observer) => {
      try {
        observer.onOrderCompleted(orderId);
      } catch (error) {
        // a falha de um observador não impede a notificação dos demais
        console.error("Observer failed while handling orderCompleted:", error);
      }
    });
  }
}

export const orderSubject = new OrderSubject();
orderSubject.subscribe(new WindowEventObserver());
orderSubject.subscribe(new LoggerObserver());
```

No page.tsx

```
const orderContext = new OrderContext(new RestApiOrderStrategy());
const orderId = await orderContext.executeOrderCreation({
        orderData,
        products,
        addOrderProductFn: addOrderProduct,
      });

      // Clear form and cart
      setCheckoutForm({
        name: "",
        lastname: "",
        phone: "",
        email: "",
        company: "",
        adress: "",
        apartment: "",
        city: "",
        country: "",
        postalCode: "",
        orderNotice: "",
      });
      clearCart();

      // AQUI EU IMPLEMENTEI O OBSERVER
      orderSubject.notifyAll(orderId);

     toast.success("Order created successfully! You will be contacted for
payment.");
```

```
      setTimeout(() => {
        router.push("/");
      }, 1000);
    } catch (error: any) {
      console.error("💥 Error in makePurchase:", error);
            if (error.message !== "DUPLICATE_ORDER" && error.message !==
"ORDER_CREATION_FAILED") {
        toast.error("Failed to create order. Please try again.");
      }
    } finally {
      setIsSubmitting(false);
    }
  };
```

4.3 Padrão 3 – Repository

- **Situação original:** os controllers acessavam o Prisma diretamente. Em `server/controllers/products.js` havia 11 chamadas `prisma.product.` (listagem admin, vitrine com filtros/sort/paginação, busca textual,CRUD), e em `server/controllers/customer_orders.js` o módulo criava o seu próprio cliente (`const prisma = new PrismaClient()`) em vez da conexão compartilhada, repetindo os 13 campos do pedido em cada `create`/`update`.
- **Problema Identificado:** mistura de regra de negócio com query de banco nos controllers; `new PrismaClient()` por módulo (vários pools); lógica de paginação, filtro e anti-duplicidade duplicada; conversão incorreta de IDs (`Number(id)` sobre UUIDs `String`); `update` copiando campos `undefined` por cima de dados existentes. Tudo isso gerava alto acoplamento ao Prisma, impossibilidade de teste unitário dos controllers e risco de regressão a cada mudança no schema.
- **Solução Implementada:** criados `ProductRepository`, `OrderRepository`, `NotificationRepository` (novo) e `UserRepository` (novo) em `server/repositories/`, todos recebendo o Prisma pelo construtor (`constructor(prismaClient = defaultPrisma)`) e exportados como singleton + classe para testes. Os três controllers migrados (`products.js`, `customer_orders.js`, `notificationController.js`) passaram a usar exclusivamente os repositórios; nenhum deles importa o Prisma. No `OrderRepository`, o `update`/`create` filtram campos `undefined` (`pickDefined`) e o `findAll` sanitiza `page`/`limit`; no `ProductRepository`, os IDs passaram a `String(id)`, foi criado o `findMany({ where, orderBy, skip, take, include })` genérico que cobre os quatro usos do controller, além de `search()` e `findRelatedOrderItems()` para a checagem de FK.

- **Trecho do código (ANTES):**

separei o código para melhor visualização

```
const prisma = require("../utills/db");
const adminProducts = await prisma.product.findMany({});
const product = await prisma.product.findUnique({
  where: { id: id },
  include: { category: true },
});
```

```
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient(); // pool próprio, além do compartilhado
const duplicateOrder = await prisma.customer_order.findFirst({
  where: { email: validatedData.email, total: validatedData.total,
           dateTime: { gte: oneMinuteAgo } }
});
const corder = await prisma.customer_order.create({
      data: {
        name: validatedData.name,
        lastname: validatedData.lastname,
        phone: validatedData.phone,
        email: validatedData.email,
        company: validatedData.company,
        adress: validatedData.adress,
        apartment: validatedData.apartment,
        postalCode: validatedData.postalCode,
        status: validatedData.status,
        city: validatedData.city,
        country: validatedData.country,
        orderNotice: validatedData.orderNotice,
        total: validatedData.total,
        dateTime: new Date()
      },
    });
```

- **Trecho do código (DEPOIS):**

```
const { productRepository } = require("../repositories");
const adminProducts = await productRepository.findMany({});
const product = await productRepository.findByIdWithCategory(id);
```

```
class ProductRepository {
  constructor(prismaClient = defaultPrisma) { this.prisma =
  prismaClient; }
  async findMany({ where = {}, orderBy = {}, skip, take, include }
  = {}) {
    return this.prisma.product.findMany({ where, orderBy,
      ...(skip !== undefined && { skip }),
      ...(take !== undefined && { take }),
      include: include || { category: { select: { name:
      true } } } });
  }
  async findById(id) {
    if (!id) return null;
    return this.prisma.product.findUnique({
      where: { id: String(id) }, // UUID: String, não Number
      include: { category: true, merchant: true, images: true } });
  }
```

```
async findByIdWithCategory(id) {
  if (!id) return null;
  return this.prisma.product.findUnique({
    where: { id: String(id) },
    include: { category: true },
  });
}

async findBySlug(slug) {
  if (!slug) return null;
  return this.prisma.product.findUnique({
    where: { slug: String(slug) },
    include: { category: true, merchant: true, images: true },
  });
}

async findFirst(where) {
  return this.prisma.product.findFirst({ where });
}
```

```
const { orderRepository, userRepository } = require("../
repositories");
const duplicateOrder = await orderRepository.findByEmailAndTotal(
  validatedData.email, validatedData.total, 1);
const corder = await orderRepository.create(validatedData);
user = await userRepository.findById(request.body.userId)
    ?? await userRepository.findByEmail(validatedData.email);
```

4.4 Padrão 4 – Factory

- **Situação original:** `server/utills/notificationHelpers.js` continha cinco funções independentes (`createOrderUpdateNotification`,`createPaymentNotification`, `createPromotionNotification`, `createSystemAlertNotification`, `createBulkNotifications`), cada uma com sua cópia do mapa título/mensagem/prioridade por status, sua chamada a `generateId()` (com `await import('nanoid')` e fallback `Math.random`) e seu `prisma.notification.create` — com `new PrismaClient()` próprio no topo do arquivo.
- **Problema Identificado:** lógica de criação copiada em cinco lugares (se o formato da notificação mudasse, cinco edições); geração de IDs inconsistente com o schema (`@default(uuid())`); `toLowerCase()` sem guarda contra `undefined`; persistência acoplada à montagem, impedindo testar a regra sem banco.
- **Solução Implementada:** criada a `NotificationFactory` (`server/factories/NotificationFactory.js`) com separação `build*` × `create*`: os builders montam o objeto puro (sem I/O) a partir dos mapas únicos `ORDER_STATUS_MESSAGES` e `PAYMENT_STATUS_MESSAGES`, com `crypto.randomUUID()` e validação central (`assertRequired`, `VALID_TYPES`, `VALID_PRIORITIES`); os métodos `create*` persistem via `NotificationRepository` (a Factory nunca importa o Prisma); o despachante `create(type, payload)` é o ponto único genérico; `setRepository()` permite injetar fake nos testes. O `notificationHelpers.js` foi reduzido de ~247 para ~61 linhas, mantendo os mesmos nomes e assinaturas como wrappers finos que delegam à Factory (compatibilidade com `customer_orders.js`).

- **Trecho do código (ANTES):**

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
const generateId = async () => {
  try { const { nanoid } = await import('nanoid'); return nanoid
  (); }
  catch (error) { return Math.random().toString(36).substr(2, 10); }
};
const createOrderUpdateNotification = async (userId, orderStatus,
orderId, totalAmount = null) => {
  const statusMessages = { 'pending': { title: 'Order Received',
    message: `Thank you! Your order #${orderId} ...`, priority:
    'NORMAL' },
    /* ... mais 5 status repetidos em cada função ... */ };
  const statusInfo = statusMessages[orderStatus.toLowerCase()] ||
  { /* fallback */ };
  const notificationId = await generateId();
  const notification = await prisma.notification.create({ data: {
    id: notificationId, userId, title: statusInfo.title, /* ...
    */ } });
  return notification;
};
```

- **Trecho do código (DEPOIS):**

```
const { randomUUID } = require('crypto');
const notificationRepository = require('../repositories/NotificationRepository');

class NotificationFactory {
  static _repository = notificationRepository;
  static setRepository(repository) { this._repository = repository; }

  static buildOrderUpdate(userId, orderStatus, orderId, totalAmount = null) {
    assertRequired(userId, 'userId');
    assertRequired(orderStatus, 'orderStatus');
    assertRequired(orderId, 'orderId');
    const template = ORDER_STATUS_MESSAGES[String(orderStatus).toLowerCase()];
    return { id: randomUUID(), userId: String(userId),
      title: template ? template.title : 'Order Update',
      message: template ? template.message(orderId)
                        : `Your order #${orderId} status has been updated to: ${orderStatus}`,
      type: 'ORDER_UPDATE', priority: template ? template.priority : 'NORMAL',
      isRead: false,
      metadata: { orderId: String(orderId), status: orderStatus,
                  ...(totalAmount != null && { totalAmount }) } };
  }
  static async createOrderUpdate(userId, orderStatus, orderId, totalAmount = null) {
    const data = this.buildOrderUpdate(userId, orderStatus, orderId, totalAmount);
    const notification = await this._repository.create(data); // sem Prisma aqui
    return notification;
  }
  static async create(type, payload = {}) { // despachante único
    switch (type) {
      case 'ORDER_UPDATE': return this.createOrderUpdate(
        payload.userId, payload.orderStatus, payload.orderId, payload.totalAmount);
      case 'PAYMENT_STATUS': /* ... */ case 'PROMOTION': /* ... */
      case 'SYSTEM_ALERT': /* ... */
      default: throw new Error(`Unknown notification type: ${type}`);
    }
  }
```

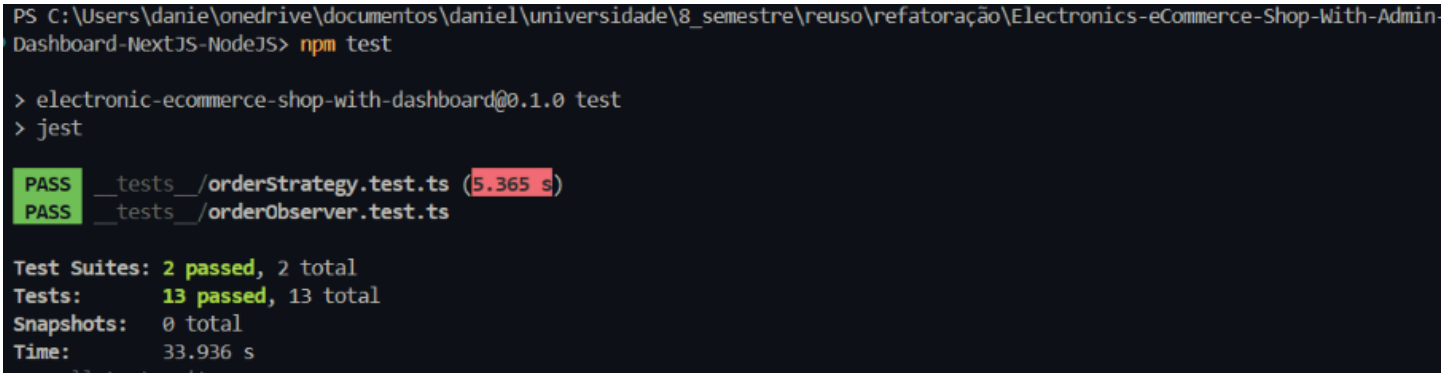
```
const { NotificationFactory } = require('../factories');
const createOrderUpdateNotification = async (userId, orderStatus, orderId, totalAmount = null) => {
  try {
    return await NotificationFactory.createOrderUpdate(userId, orderStatus, orderId, totalAmount);
  } catch (error) { console.error('❌ Error creating order notification:', error); throw error; }
};
```

5. Análise dos Resultados

5.1 Strategy e Observer

ASPECTO	ANTES	DEPOIS
Acoplamento	makePurchase dependia diretamente de chamadas	Página depende das abstrações OrderContext e

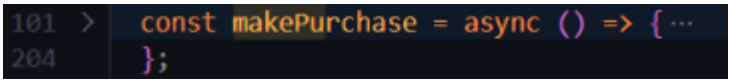
	HTTP para criar o pedido e de window.dispatchEvent	orderSubject
Coesão	Um manipulador acumulava validação de UI, protocolo REST, loop de produtos e eventos	Cada classe tem uma responsabilidade (criar pedido, notificar, registrar log)
Extensibilidade	Novo fluxo de pedido exigia editar a página; nova reação exigia nova chamada na página	Nova estratégia via setStrategy; novo observador via subscribe, sem tocar em makePurchase
Testabilidade	Lógica presa ao componente React, sem testes	13 testes unitários em Jest cobrem a estratégia, o contexto e o observer
Legibilidade	Bloco longo com muitos console.log e tratamento de erro aninhado	makePurchase reduzido em 105 linhas (de 209 para 104). A lógica de criação do pedido foi deslocada para OrderStrategy.ts, não eliminada.



Antes:



Depois:



Os resultados mostram melhoria nos aspectos avaliados, com algumas ressalvas. Redução de duplicação: no código original, o tratamento das respostas de erro da API aparecia em dois pontos de makePurchase (no bloco !response.ok e novamente no catch), inclusive com o mesmo laço de exibição das mensagens de validação repetido. Após a refatoração, esse tratamento existe em um único lugar, o método createOrder, e o catch da página ficou restrito a evitar mensagens duplicadas.

Facilidade de manutenção: mudanças no protocolo de criação do pedido (rota, formato de erro) agora afetam uma classe, e não o componente de interface, e os 13 testes unitários que passam (evidência acima) protegem esse comportamento contra regressões. Reutilização de componentes: OrderContext e orderSubject não dependem do checkout, de modo que podem ser reaproveitados por outras telas, como a criação de pedidos pelo painel administrativo. No estado atual, porém, apenas o checkout os utiliza, então esse ganho ainda é potencial. Organização e legibilidade: a página passou a descrever o fluxo (validar, criar o pedido, limpar o estado, notificar), enquanto os detalhes ficaram nas classes. Como limitação, o total de código aumentou, já que a lógica foi movida e foram adicionadas interfaces e classes, e a estratégia ainda depende de toast, uma responsabilidade de interface dentro de lib/.

5.2 Factory e Repository

ASPECTO	ANTES	DEPOIS
Acoplamento	Controllers chamavam prisma.* direto; 3 módulos criavam new PrismaClient() próprio	Controllers dependem dos repositories; Prisma único via utills/db.js (12 testes provam a delegação com Prisma fake)
Coesão	Paginação, filtro e anti-duplicidade escritos no meio dos controllers	Cada repository tem uma responsabilidade (produtos, pedidos, notificações, usuários); cada suíte de teste cobre uma classe
Extensibilidade	Mapas de mensagem e generateId copiados em 5 funções; 13 campos do pedido repetidos em create/update	Mapas únicos na Factory; pickDefined central; helpers de ~247 para ~61 linhas (2 testes provam que os helpers só delegam)
Testabilidade	0 testes no backend; tudo exigia MySQL ligado	28 testes em ~235 ms sem banco (8 de builders puros + 5 de persistência com mock + 15 de repositories com Prisma fake)
Legibilidade	Number(id) sobre UUID; update com undefined; toLowerCase() sem guarda	Testes que falhariam no código antigo: String(id), pickDefined, assertRequired e enums validados

Os resultados mostram melhoria nos aspectos avaliados, com algumas ressalvas. Redução de acoplamento: antes, testar qualquer controller exigia MySQL ligado, pois o Prisma concreto estava embutido no código; depois, os 15 testes de repositories passam um Prisma fake pelo construtor e os 7 testes de persistência da Factory usam setRepository(), o que só é possível porque os consumidores dependem de abstrações. Melhoria da coesão: cada suíte cobre uma única classe com uma única responsabilidade — a suíte de builders, por exemplo, valida somente montagem (título, prioridade e metadata por status), sem nenhum I/O. Redução de duplicação: os dois testes de delegação dos helpers espionam a Factory e comprovam que as funções antigas não contêm mais

lógica própria (argumentos repassados intactos e uma única persistência por chamada), evidenciando a eliminação das ~200 linhas copiadas. Facilidade de manutenção: os testes de String(id), pickDefined e validação de enums documentam como executable specification os três bugs reais encontrados na duplicação — se qualquer um deles for reintroduzido, ao menos um teste quebra. Extensibilidade: o teste do despachante create(type, payload) fixa o contrato de que um tipo novo exige apenas um build* e um case, e o teste de tipo desconhecido garante falha explícita em vez de comportamento silencioso. Reutilização de componentes: NotificationRepository é compartilhado pela Factory e pelo notificationController, e UserRepository atende a pedidos e notificações, ambos exercitados indiretamente pelos testes de delegação. Organização e legibilidade: os nomes das suítes e dos testes descrevem o comportamento esperado do sistema ("update filtra campos undefined", "buildBulk gera um item por usuário"), servindo como documentação viva.

Como limitações, os testes usam mocks e verificam contrato e fiação (wiring), não o comportamento real contra o MySQL — a integração com o banco continua sem cobertura automatizada. O notificationController.js migrado não possui teste direto, apenas o repositório que ele consome. A migração para repositories cobriu produtos, pedidos e notificações; os demais controllers (category, merchant, users etc.) seguem com Prisma direto e fora do escopo. Por fim, como nas refatorações do frontend, o total de código aumentou (quatro arquivos novos de repository mais dois de teste), custo consciente em troca de desacoplamento e testabilidade.

6. Conclusão

- **Refatorações**:

  Foram aplicados três padrões GoF e o padrão Repository. No cliente, o Strategy (RestApiOrderStrategy e OrderContext) isolou a criação do pedido do componente de checkout, e o Observer (OrderSubject, WindowEventObserver e LoggerObserver) desacoplou as reações à conclusão do pedido. No servidor, o Repository (ProductRepository, OrderRepository, NotificationRepository e UserRepository) centralizou o acesso ao Prisma, e a Factory (NotificationFactory) unificou a criação de notificações, antes espalhada em cinco funções.

- **Benefícios:**

  makePurchase passou de 209 para 104 linhas, e a lógica de criação do pedido ganhou uma classe própria, coberta por 13 testes unitários em Jest. Nos controllers, as chamadas diretas ao Prisma foram substituídas pelos repositórios, e os vários new PrismaClient() deram lugar a uma conexão compartilhada. O notificationHelpers.js foi reduzido de cerca de 247 para cerca de 61 linhas, eliminando a duplicação dos mapas de mensagens e da geração de IDs. A mudança também permitiu corrigir problemas latentes, como a conversão de IDs com Number(id) sobre UUIDs e atualizações que copiavam campos undefined.

- **Contribuições dos padrões para o reuso e evolução do sistema:**

  o Strategy permite acrescentar novas formas de criar pedidos (outro protocolo ou um mock para testes) sem alterar a página, e o Observer permite registrar novas reações com subscribe, sem tocar em makePurchase. Como apenas o checkout os utiliza hoje, esse ganho ainda é potencial. No servidor, o Repository concentra as consultas em um único lugar, de

modo que uma troca de ORM ficaria restrita aos arquivos de repositório, e a Factory exige apenas um novo build* e um novo case para cada tipo de notificação. Em ambos, a injeção do cliente Prisma ou do repositório permite substituí-los por objetos falsos em testes.

- **Dificuldades e limitações:**

  A estratégia ainda exibe toast, que é uma responsabilidade de interface dentro de lib/, e a busca do userId e a montagem do orderData continuam na página. Antes do LoggerObserver, o Observer tinha um único consumidor, o que enfraquecia sua justificativa. A NotificationFactory usa um despachante estático com switch, aproximando-se mais de uma fábrica simples do que do Factory Method clássico do GoF, e o notificationHelpers.js foi mantido como camada de compatibilidade, o que preserva uma indireção a mais. Por fim, as correções de IDs e de atualizações alteram o comportamento anterior de forma intencional.

- **Aprendizados:**

  O principal aprendizado foi perceber que a escolha de um padrão deve partir do problema concreto do código, e não do nome do padrão. No sistema real, os sinais que justificaram cada escolha eram observáveis: um manipulador de cerca de 200 linhas misturando interface e protocolo HTTP (Strategy); uma ação com efeitos colaterais disparada diretamente na página (Observer); consultas ao Prisma e vários new PrismaClient() espalhados pelos controllers (Repository); e cinco funções de notificação com a mesma lógica copiada (Factory). Aplicar os padrões a esse contexto tornou concretos conceitos que até então eram teóricos.

  Os ganhos, porém, não são uniformes. Onde havia duplicação, o código diminuiu (o notificationHelpers.js passou de cerca de 247 para cerca de 61 linhas). Já Strategy e Observer aumentaram o total de código, pois introduziram interfaces e classes; o benefício neles é a extensibilidade, com nova estratégia ou novo observador sem alterar a página. Percebemos ainda o risco de excesso de engenharia: a justificativa do Observer só ficou clara ao adicionarmos um segundo observador.

  Também foi interessante perceber que padrões de projeto indicam uma preocupação não apenas com o estado atual do código, mas também com o que eles podem se tornar. Afinal, os padrões apresentam alternativas para quando o projeto crescer e a necessidade de novas features se tornarem necessárias. Além disso, a percepção de como implementar um padrão em um projeto de médio/grande porte torna a teoria mais prática e concreta, é possível enxergar com maior facilidade os efeitos causados por eles.

  Ressaltamos que fizemos uso de IA (Claude e Gemini) para entender o contexto do projeto, validar se os padrões escolhidos estavam implementados corretamente e ajudar com a configuração dos testes e compreensão técnica do código.

7. Referências

https://refactoring.guru/pt-br